ReqMind AI — Compte rendu d'équipe

Analyse automatique d'exigences logicielles

Nom et prénom : **Zakariae CHELLE** Groupe : **5IASDT G1**
Module : **Traitement automatique du langage (NLP)**
Équipe : **1** · Mission : **ReqMind AI** Référent du dépôt : **CHELLE Zakariae**
Membres : **CHELLE Zakariae (A)** , **ISAKROUHEN Yasmine (B)**

Les réponses portent sur le travail effectué pendant la séance.

Cadrage du projet

1 Besoin et utilisateur

Reformulez le problème et indiquez qui utilisera la solution.

Réponse : Analyser des exigences logicielles rédigées librement. Utilisateurs : développeurs et analystes fonctionnels.

Entrée finale : texte ou fichier TXT. Sortie : exigences extraites, catégories, ambiguïtés et doublons.

2 Entrées et sorties

Décrivez les entrées et les sorties du projet final. Distinguez-les du résultat de prétraitement attendu aujourd'hui.

Réponse : Aujourd'hui : phrases segmentées et prétraitées, avec comparaison avant/après.

Dataset envisagé : PROMISE (exigences logicielles, anglais ; format textuel/CSV selon version).

3 Fonctionnalités prioritaires

Fonctionnalité	Critère d'acceptation observable
Extraction / segmentation	Une exigence par segment dans la liste.
Classification	Étiquette fonctionnelle ou non fonctionnelle affichée.
Ambiguïtés et doublons	Expressions vagues et paires similaires signalées.

Données et exemples

4 Source et données chargées

URL, nom/version du corpus, langue, format, champs utiles, nombre d'éléments effectivement chargés et conditions d'utilisation.

Réponse : URL/version, champs, licence et nombre chargés : à vérifier ; aucun chargement attesté à ce stade.

Annotations existantes : types d'exigences (selon la version PROMISE retenue).

5 Annotations

Quelles annotations existent ? Lesquelles faudra-t-il créer ? Si votre tâche n'a pas de classes, indiquez-le.

Réponse : Existantes : classes d'exigences du corpus PROMISE (FR et types de NFR, tableau ci-dessous).

À créer : exemples annotés pour ambiguïtés et doublons ; protocole de validation en binôme.

Classes d'exigences utilisées pour la classification :

Code	Signification	Explication
FR	Functional Requirement	Exigence fonctionnelle : ce que le système doit faire.
NFR	Non-Functional Requirement	Exigence non fonctionnelle : comment le système doit fonctionner.
PE	Performance	Exigences de performance : rapidité, temps de réponse, débit.
LF	Look and Feel	Apparence et ergonomie : interface, design, expérience utilisateur.
US	Usability	Facilité d'utilisation et d'apprentissage.
SE	Security	Sécurité : authentification, confidentialité, protection.
A	Availability	Disponibilité du système.
MN	Maintainability	Facilité de maintenance et de modification.
O	Operational	Contraintes liées à l'environnement d'exploitation.
L	Legal	Contraintes légales et réglementaires : licences, conformité, protection des données.
SC	Scalability	Capacité à supporter une montée en charge : plus d'utilisateurs, plus de données.
FT	Fault Tolerance	Capacité à continuer de fonctionner malgré une panne ou une erreur.
PO	Portability	Capacité à fonctionner sur différentes plateformes ou environnements.

> ***Remarque :*** *FR et NFR forment la classification principale (2 classes) ; les codes PE à PO détaillent les sous-types de NFR. Exemples 1 et 3 : FR ; exemple 2 : NFR (ambiguë) ; exemple 4 : NFR – PE.*

6 Cinq exemples commentés

Au moins trois exemples proviennent du corpus si celui-ci est accessible. Identifiez les exemples que vous construisez. Incluez deux cas difficiles. La sortie finale est une proposition à justifier ou une annotation existante, jamais une prédiction déjà calculée sans modèle.

Cas et origine	Entrée et sortie finale attendue	Difficulté ou justification
1 — construit	« L'utilisateur doit se connecter. » → fonctionnelle.	Action utilisateur identifiable.
2 — construit	« Le site doit être rapide. » → non fonctionnelle, ambiguë.	« rapide » non mesurable.
3 — construit	« Le système ne doit pas supprimer les comptes. » → fonctionnelle.	Négation à préserver.
4 — construit (difficile)	« La page charge en 2 s. » → performance.	Nombre et unité essentiels.
5 — construit (difficile)	« Un admin peut effacer un compte » / « Un admin peut supprimer un compte » → doublon possible.	Formulations différentes, sens proche.

Prétraitement et résultat

7 Décisions de prétraitement

Élément	Décision	Justification et risque
Casse	Conserver casse d'origine ; version minuscule pour TF-IDF.	La casse peut aider les modèles contextuels.
Nombres et dates	Conserver nombres, dates et unités.	Critères mesurables ; suppression risquée.
Ponctuation et URL	Nettoyer espaces ; garder ponctuation utile et URL.	Marqueurs de sens et références.
Négation et mots fréquents	Garder négations et mots-outils.	Éviter les inversions de sens.
Structure et annotations	Préserver frontières de phrases et labels.	Traçabilité des exigences.

8 Résultats exécutés

Indiquez les cellules ou le fichier où figurent les cinq résultats avant/après. Expliquez une information préservée et une perte éventuelle.

Réponse : Non exécuté à ce stade : joindre ensuite `preprocessing_examples.ipynb` avec 5 cas avant/après.

À préserver : négation et nombres. Risque : perte de ponctuation utile au sens.

9 Comparaison en binôme

Chacun décrit sa variante. Comparez-les sur les mêmes exemples. Expliquez le choix commun sans annoncer de gain de performance d'un futur modèle.

Réponse : Variante A : nettoyage léger ; variante B : normalisation plus forte. Comparaison à réaliser sur les mêmes 5 phrases.

Choix provisoire : nettoyage léger afin de conserver les indices sémantiques.

Conception et organisation

10 Schéma provisoire

Insérez ou dessinez les étapes. Identifiez les composants déjà réalisés et les composants futurs.

Réponse : Texte → segmentation → nettoyage → classification → ambiguïtés / similarité → interface Streamlit.

Réalisé : cadrage du pipeline. À réaliser : chargement, prétraitement, modèles et interface.

11 Décision personnelle

Quel choix de l'équipe sert votre mission ? Quel exemple permet de le justifier ?

Réponse : Décision : préserver les négations ; « le compte ne doit pas être supprimé » changerait de sens sans « ne pas ».

Autre décision : conserver chiffres et unités pour tester les exigences mesurables.

12 Répartition et prochaine action

Membre	Travail réalisé	Prochaine tâche
Étudiant A — CHELLE Zakariae	Cadrage de la classification (prévu)	Vérifier PROMISE, charger les données et préparer TF-IDF.
Étudiant B — ISAKROUHEN Yasmine	Cadrage ambiguïtés/doublons (prévu)	Construire 5 exemples, tester normalisation et embeddings.

13 Difficultés rencontrées

Décrivez le blocage, la tentative de résolution et la suite. En cas de problème de données, joindre le message exact.

1. **Choix de la version du corpus PROMISE**

   Blocage : plusieurs copies de PROMISE circulent (dépôt PROMISE d'origine, Zenodo, Kaggle, GitHub), dans des formats différents (ARFF, CSV) et avec un nombre d'exigences et des étiquettes qui varient selon la version. Nous ne savons pas encore laquelle sert de référence ; c'est pourquoi aucun chargement n'est attesté en section 4.

   Message : pas de message d'erreur : le blocage porte sur le choix de la source, pas sur le chargement lui-même.

   Tentative : comparaison des sources disponibles ; choix provisoire de la version « PROMISE NFR », qui distingue exigences fonctionnelles (F) et types non fonctionnels (performance, sécurité, utilisabilité…).

   Suite : valider la version avec l'enseignant, puis renseigner en section 4 l'URL, la licence, les champs et le nombre d'éléments réellement chargés.

2. **La liste de mots vides supprime la négation**

   Blocage : la liste de mots vides français de NLTK (`stopwords.words("french")`) contient « ne » et « pas ». Appliquée à l'exemple 3, elle inverse le sens de l'exigence :

Avant	`Le système ne doit pas supprimer les comptes.`
Après	`système doit supprimer comptes`

   Tentative : liste personnalisée qui retire des mots vides les marqueurs de négation : « ne », « pas », « jamais », « aucun », « sans », « ni ».

   Suite : vérifier le résultat sur les 5 exemples dans `preprocessing_examples.ipynb` (décision de la section 7).

3. **La segmentation coupe les nombres décimaux**

   Blocage : une découpe naïve sur le point (`texte.split(".")`) coupe une exigence mesurable en deux et fait perdre le critère (cas de l'exemple 4 avec une valeur décimale) :

Avant	`La page charge en 2.5 s. Le site doit être rapide.`
Après	`['La page charge en 2', '5 s', ' Le site doit être rapide', '']`

   Tentative : expression régulière qui ne coupe qu'après un point suivi d'un espace et d'une majuscule, ou segmenteur de phrases de spaCy.

   Suite : tester sur des exigences contenant des nombres, des versions (« v1.2 ») et des abréviations (« etc. », « ex. »).

4. **Doublons et contradictions mal repérés par TF-IDF**

Blocage : la similarité TF-IDF dépend des mots partagés, pas du sens. Le synonyme « effacer » / « supprimer » (exemple 5) baisse le score, alors que deux phrases contradictoires obtiennent un score élevé :

Paire 1	`« Un admin peut effacer un compte » / « Un admin peut supprimer un compte » → doublon`
Paire 2	`« Le compte doit être supprimé » / « Le compte ne doit pas être supprimé » → contradiction`

Tentative : comparer, sur ces paires, TF-IDF + cosinus et des embeddings de phrases (modèle multilingue, car le corpus est en anglais et nos exemples en français).

Suite : annoter en binôme une dizaine de paires (doublon / non doublon / contradiction) pour fixer un seuil de similarité justifié.

> ***Risque principal :*** *disponibilité et version des données PROMISE (difficulté 1) ; les autres points sont traités par les décisions de prétraitement de la section 7.*